

Chapter 2

OVERVIEW OF JAVA LANGUAGE



INDEX

SL. No.	Topic	Page No.
1	<u>Java's Two Faces</u>	3
2	<u>Anatomy of a Java Statement</u>	5
3	<u>Structure to Execution</u>	7
4	<u>Code Comments</u>	8
5	<u>First Program (Line-by-Line Analysis)</u>	9
6	<u>Bring Java to Life</u>	10
7	<u>Write Once, Run Anywhere: The Magic of Bytecode</u>	11
8	<u>Java Virtual Machine (JVM)</u>	12
9	<u>Command Line Arguments</u>	12
10	<u>The Factorial Program</u>	14
11	<u>Best Practices for Readable Code</u>	16
12	<u>PATH vs. CLASSPATH</u>	16

OVERVIEW OF JAVA LANGUAGE

Master the Basics: An Illuminated Guide to Java Fundamentals

Java is a general-purpose, object-oriented programming language designed to create robust and versatile software. To understand how Java works, we must first look at the two primary types of programs it can produce and the journey those programs take from a developer's mind to a working application.

Java's Two Faces: Applications vs. Applets

While Java is a single language, it manifests in two distinct forms depending on where the code is intended to run and what it is meant to accomplish.

Feature	Java Applications	Java Applets
Purpose	Stand-alone programs written to carry out specific tasks.	Small programs developed specifically for Internet applications.
Location	Stored and executed on a local computer.	Located on a distant computer and downloaded via the Internet.
Execution Environment	Executed using the Java interpreter (<code>java</code>).	Executed within a Java-capable web browser.
Common Tasks	Solving equations, reading/writing files, and system management.	Providing interactive elements or widgets within a webpage.

The "Desktop Calculator vs. Interactive Web Widget" Analogy Think of a **Java Application** as a professional desktop calculator sitting on your desk; it is a stand-alone tool you use for local tasks like balancing a budget. In contrast, a **Java Applet** is like an interactive currency converter widget on a travel website; it is a small, specialized tool that lives on the web and runs only when you visit its page through your browser.

Understanding which type of program you are building determines how that code is prepared and delivered to the user.

The Compilation Journey: From Source to Output

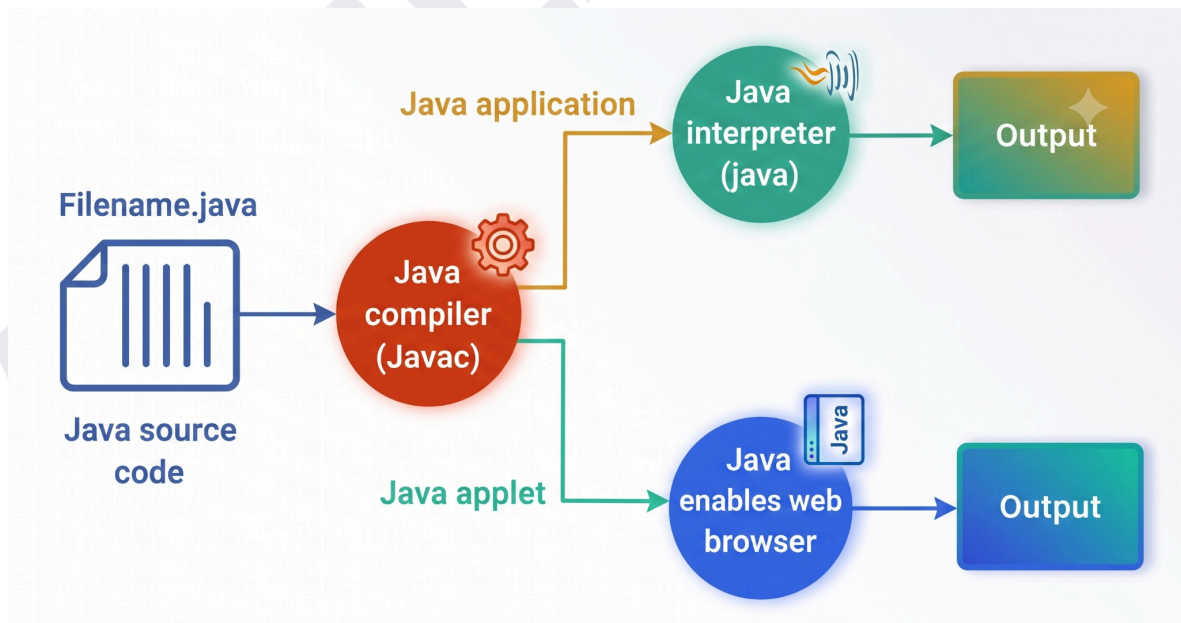
Java uses a unique two-step execution process that ensures code can run on different types of systems. This journey involves translating human-readable text into a format the computer can understand.

1. **Source Code:** The journey begins with the **Filename.java** file. This is the raw text written by the programmer.
2. **The Compiler (javac):** The Java compiler takes the source code and translates it into an intermediate form called **byte code**.
3. **The Executor:** This is where the path splits based on the program type:
 - **For Applications:** The byte code is processed by the **Java Interpreter (java)** to produce the final output.
 - **For Applets:** The byte code is handled by a **Java-enabled web browser** to display the output to the user.

The "Foreign Recipe" Analogy

- The **Source Code** is like a recipe written in a foreign language that you cannot read.
- The **Compiler** is a translator who turns that text into a series of simple pictures (**Byte Code**) that anyone, regardless of their language, can follow.
- The **Interpreter/Chef** is the person who follows those pictures to finally cook the meal (**Output**).

Before we can compile an entire program, however, we must understand the individual "sentences" that make up the source code.



Anatomy of a Java Statement

In Java, a **Statement** is an executable combination of tokens ending with a semicolon (;). It represents a single action or command that the computer must perform. Statements are typically executed one after another in a sequential order.

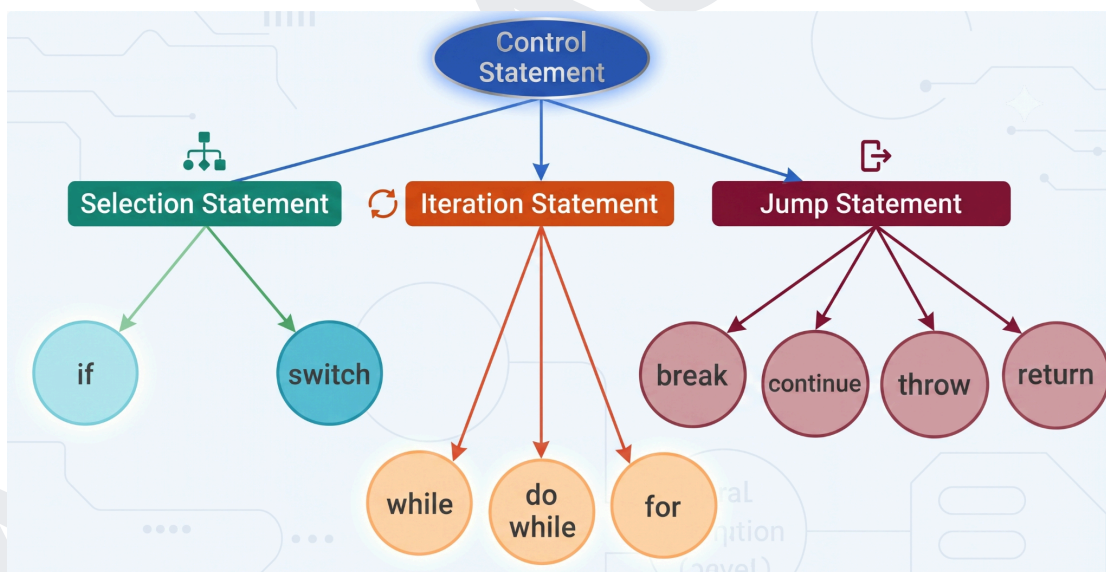
The most vital technical detail of a statement is the **semicolon**. In Java, every statement must end with this character to be considered complete.

Analogy: Just as an English sentence ends with a period to signal the completion of a thought, a Java statement ends with a semicolon to signal the completion of a command to the computer.

While there are many types of statements, **Expression Statements** are the most common. According to technical standards, these include:

- **Assignments:** Setting a value to a variable.
- **Increments/Decrements:** Specifically **Pre-Increment**, **Pre-Decrement**, **Post-Increment**, and **Post-Decrement**.
- **Method Calls:** Triggering a specific function or action.
- **Allocation Expressions:** Creating new objects or space in memory.

While statements usually run in a straight line, we often need to redirect the computer's "train of thought" using specialized control tools.



Directing the Flow: Control Statements

Control statements are the "brain" of your code. They allow the program to move beyond simple top-to-bottom execution, enabling it to make decisions and repeat actions. Without these, software would be unable to handle dynamic user input or changing conditions.

- **Selection Statements (Branching):**
 - **Commands:** `if`, `switch`.
 - **Analogy:** The **Fork in the Road**. Based on a specific condition (like "Is it raining?"), the program chooses one path over another.
- **Iteration Statements (Looping):**
 - **Commands:** `while`, `do while`, `for`.
 - **Analogy:** The **Factory Conveyor Belt**. The program repeats the same task over and over until the job is finished or a specific condition is met.
- **Jump Statements:**
 - **Commands:** `break`, `continue`, `throw`, `return`.
 - **Technical Function:** These pass control to the beginning or end of a current block, or to a labeled statement.
 - **Analogy:** The **Board Game Shortcut**. These allow the program to skip ahead, start a turn over, or jump to an entirely different section of the "board."

By using these tools, a developer transforms a static list of instructions into dynamic logic that can respond to different situations. We now move from these common logic tools to the specialized statements used for specific programming scenarios.

The Specialized Toolset

Beyond the standard flow of logic, Java provides specialized statements for specific programming scenarios. These ensure the program remains organized, safe, and efficient.

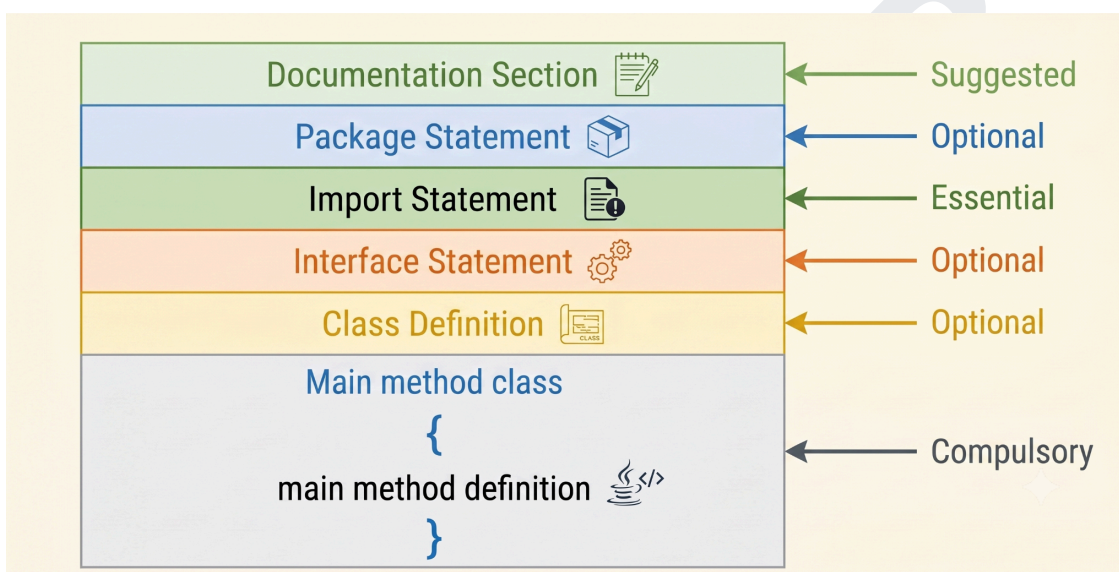
- **Empty Statements:** Used as a **placeholder** during program development; they do nothing.
 - *Analogy:* Like a **Sticky Note** left on a page to remind you to fill in the details later.
- **Labeled Statements:** Used for **identification**, allowing any statement to be tagged with a name. **Technical Note:** Labels must not be keywords, already declared local variables, or previously used labels in the same module.
 - *Analogy:* Like a **Bookmark** used to find and jump to a specific spot in a large book quickly.
- **Guarding Statements:** Used for the **safe handling** of code that might cause exceptions (errors). These statements utilize the keywords `try`, `catch`, and `finally`.
 - *Analogy:* Like a **Safety Net** that catches a circus performer if they happen to fall during an act.
- **Synchronization Statements:** Used for handling issues in **multithreading** (when a program performs multiple tasks simultaneously). These prevent data corruption by ensuring threads do not interfere with one another.
 - *Analogy:* Like a **Traffic Light** that prevents multiple cars (data threads) from crashing into each other at an intersection.

From Structure to Execution

To the uninitiated, computer code can look like a chaotic jumble of characters; however, to a developer, it is a meticulously organized blueprint. Mastering Java begins with understanding its architectural layout—the specific structure that allows the computer to transform a text file into a functional application.

The Anatomy of a Java Program (The Building Blocks)

A Java program is a highly structured document composed of distinct sections. While some components are optional tools used to organize complex projects, others are the essential foundation upon which all execution rests.



Section Name	Status	Purpose
Documentation Section	Suggested	Contains comment lines providing the program name, author, and metadata.
Package Statement	Optional	Declares a namespace to inform the compiler which group the classes belong to.
Import Statement	Required (if using libraries)	Brings in external classes or built-in packages so the code can utilize them.
Interface Statement	Optional	Defines a collection of abstract methods for classes to implement.
Class Definition	Optional	Maps real-world problems into code; a program may contain multiple definitions.
Main Method Class	Compulsory	The essential entry point where the program's execution actually begins.

The "Front Door" of the Application

Think of a Java program as a complex house. While a house may have many rooms (classes) and windows (methods), an outside visitor—the Operating System—requires a specific "front door" to enter. The **Main Method Class** serves this purpose. Because every stand-alone Java program requires the `main()` method as its starting point, the system enters here, creates the necessary objects, and facilitates communication between them. On reaching the end of the `main()` method, the program "exits" the house and terminates.

Standard Section Roles

- **Documentation:** Provides human-readable context, such as the program's intent or history.
- **Package:** The very first allowed statement, used to categorize the file within a larger project.
- **Import:** Acts as a bridge, allowing the program to use code stored in other files or libraries.
- **Interface:** Similar to a class, it acts as a template for inheriting abstract methods.
- **Class Definition:** The primary building blocks of Java; these model the problem you are trying to solve.
- **Main Method Class:** Contains the `main()` definition, holding the logic that the computer is instructed to execute first.

Understanding this layout allows us to communicate with other developers through strategic notes, but to communicate with the computer itself, we must master the specific grammar of the language.

Code Comments: The Developer's Sticky Notes

Comments are non-executable lines used to provide explanations for humans. In the world of programming, they are the "chef's recipe notes" that ensure a recipe can be followed by others—or by yourself months later.

- **Single-line comments (`//`):** These function like a **quick margin note**. Use them for brief explanations of a single line of logic.
- **Multi-line comments (`/* ... */`):** These act as **detailed technique explanations**. They are ideal for describing complex algorithms that span several lines.
- **Documentation comments (`/** ... */`):** These are **formal notes for a professional cookbook**. This special format is used by tools to generate professional API documentation automatically.

[!NOTE] **The Importance of Readability** Comments are vital for code maintenance. They explain the "why" behind a block of code, ensuring that the logic remains clear during debugging or future updates.

Deconstructing Your First Program (Line-by-Line Analysis)

To understand Java, we perform a "grammatical parsing" of the code. Each keyword serves a specific role in the programmatic "sentence."

```
// Program to find square of a number
class square
{
    public static void main(String args[])
    {
        int a=5;
        double b;
        b=Math.pow(a,2);
        System.out.println ("Square of \t" + a + "\tis \t " + b);
    }
}
```

Program Output

Square of 5 is 25.0

The Keyword Dictionary

- **public (Adjective):** An access specifier describing visibility. It tells the compiler that this section is accessible to all other classes.
- **class (Noun):** The category or entity being defined. It is the fundamental unit of Java programming.
- **static (Modifier):** A keyword that indicates a method belongs to the class itself rather than a specific instance.
- **void (Adjective):** Describes the return type, signifying that this specific "verb" (method) performs an action but returns no value.
- **main (The Entry Verb):** The most important method in the program. According to the source, a class without a **main** method will result in a compilation error when you attempt to run it.
- **System.out.println (Action Statement):** The standard output command used to print results to the console.
- **Braces {} (Punctuation):** These define the boundaries of the code "paragraphs." Every class and method begins with { and must close with }.

Mathematical Parsing & Formatting

In a program calculating the square of a number, we see these roles in action:

- **Declaration:** `int a=5;` creates a variable (noun) and assigns it a value.
- **Invocation:** `b=Math.pow(a,2);` calls the `pow` method from the built-in `Math` class to perform the calculation.

- **Concatenation & Escaping:** In the statement `System.out.println ("Square of \t" + a + "\t is \t" + b);:`
 - The **+** symbol acts as a concatenation operator, joining text strings with numerical variables.
 - The **\t sequence** is a "tab" escape character, which inserts a horizontal space to format the output cleanly.

Once you have mastered the grammar, you are ready to move from writing text to bringing the code to life.

The Three Steps to Bring Java to Life

Developing a Java application follows a strict three-step workflow: **Create, Compile, and Run.**

1. **Writing (.java):** This is the **Composition**. You use a text editor to write your instructions and save them with a `.java` extension.
2. **Compiling (javac):** This is the **Publisher**. You use the `javac` tool to transform your human-readable "song" into **Bytecode**.
3. **Running (java):** This is the **Performer**. The Java interpreter reads the bytecode and "plays" the instructions on your specific instrument (the local machine).

[!WARNING] **The Critical Matching Rule** For the compiler to find your code, the source file name **MUST** exactly match the name of the public class. If your code is `class CS`, your file must be saved as `CS.java`.

Command Line Execution

Use these exact commands to move from source code to a running program:

Step 1: Compile

- `javac CS.java`

This creates `CS.class`, an intermediate file containing bytecode.

Step 2: Run

- `java CS`

*Note that we do **not** include the `.class` extension here. The interpreter automatically looks for the class file and executes it.*

This multi-step process is the secret behind Java's most famous capability: platform independence.

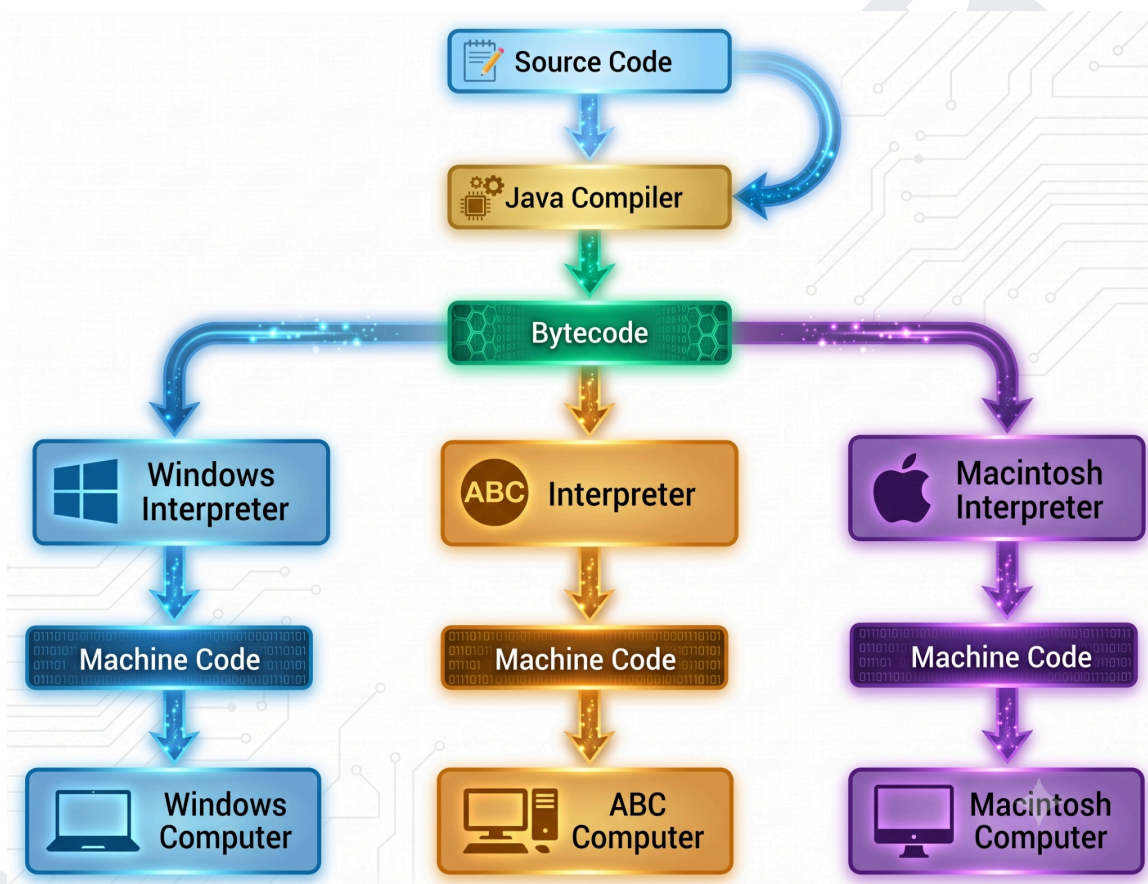
Write Once, Run Anywhere: The Magic of Bytecode

Java's "Platform Independence" is achieved through an intermediate language called **Bytecode**. This allows a single file to run on any operating system without being rewritten.

The "IKEA Assembly Manual" Analogy

Imagine an assembly manual that uses only universal pictures (Bytecode). Whether the person building the furniture is in New York, Tokyo, or Paris (Windows, Mac, or Linux), they can follow the same pictures. However, they need a **local worker (the Interpreter)** who speaks their local language (Machine Code) to actually assemble the parts.

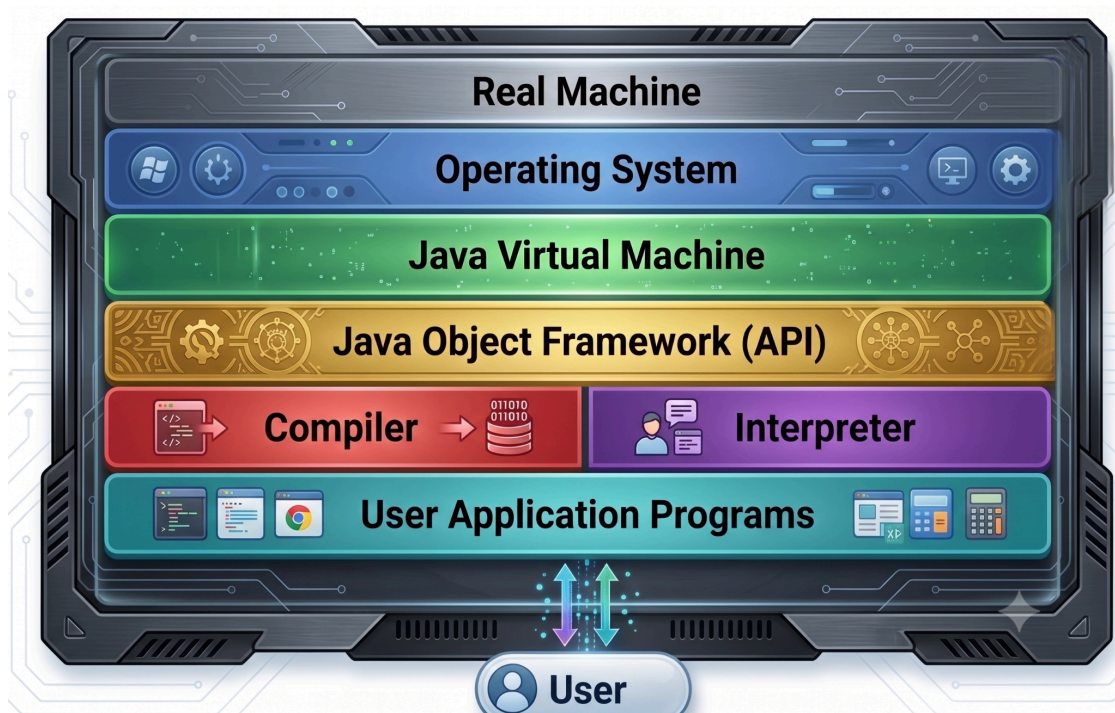
The Execution Flow



1. **Source Code:** The high-level text written by the programmer.
2. **Java Compiler (javac):** Translates the source into machine-independent **Bytecode**.
3. **Bytecode:** The universal, intermediate instructions.
4. **Java Interpreter (java):** A tool specifically built for each operating system (Windows, Mac, etc.) that reads the Bytecode.
5. **Machine Code:** The final, hardware-specific instructions generated by the interpreter.

By decoupling the writing of code from the specifics of the hardware, Java fulfills its primary philosophy: **"Write Once, Run Anywhere."** Once you have generated the bytecode, any machine equipped with a Java interpreter can bring your architectural blueprint to life.

The Java Virtual Machine (JVM): The Software Computer



- **Concepts:** The JVM as an abstract computer living in memory. We will explain the compilation process (Source Code → Bytecode) and interpretation process (Bytecode → Machine Code via platform-specific interpreters), as well as the interactive layering diagram.
- **Real-World Analogy: An Automatic Translation Earpiece.** You speak in English (Source Code), which is recorded into a standardized transcript (Bytecode). That transcript is read by specialized local modules that speak aloud in German, Spanish, or Japanese depending on who is listening (machine-specific interpreters).

Command Line Arguments

In the Java ecosystem, **command line arguments** are specific parameters supplied to an application program at the moment of invocation. Unlike interactive inputs that prompt a user while the program is already running, these arguments are passed through the operating system's terminal or command prompt as the program is being launched.

Concept at a Glance The primary benefit of command line arguments is efficiency. They allow a program to receive **quick inputs** and configuration data immediately, bypassing the need for complex graphical interfaces or interactive text prompts during execution.

To utilize these external inputs, the Java Virtual Machine (JVM) requires a structured storage location where data is captured the instant the program initializes.

The Storage Vessel: Understanding the `args[]` Array

Every standard Java application begins execution at the `main` method. This method acts as the entry point and the primary collection area for external parameters. These inputs are stored in a specialized structure known as an **array**. Specifically, the `args[]` parameter in `public static void main(String args[])` is the vessel for this data.

The Nature of `args[]`

Feature	Description
Data Type	An array of Strings .
Content	A sequential collection of strings provided by the user at the command line.
Exclusion	The file name of the program is NOT included in this list.

Architect's Note: It is important for developers transitioning from languages like C or C++ to note a key distinction: in Java, `args[0]` is the first actual argument provided by the user. Java does not include the executable name in the array, ensuring the storage vessel contains only the data you intended to process.

Once the inputs are stored in this array, we navigate the data using a coordinate system known as indexing.

The Map to Your Data: Zero-Based Indexing

To retrieve specific values from the `args[]` array, we use **subscripts** or **indexes**. In Java, the index `i` (written as `args[i]`) denotes the exact position of a piece of data within the container.

Java utilizes **zero-based indexing**, meaning the first element is at position 0. It is a common beginner error to assume `args[0]` is a number; however, at this stage, every element is stored as a **String object**. Using the example of passing the inputs **5, 3, and 4**, the mapping is as follows:

- `args[0]` → The first element (the String `"5"`).
- `args[1]` → The second element (the String `"3"`).
- `args[2]` → The third element (the String `"4"`).

Understanding this mapping is the key to writing logic that can iterate through user data programmatically.

Case Study: The Factorial Program Breakdown

The following program demonstrates how to process multiple command line arguments to calculate factorials.

Note: While the source uses the class name `factorial`, standard Java naming conventions typically dictate PascalCase (e.g., `Factorial`).

```
// Program to find factorial of list of number reading from command line
argument
import java.io.*;
public class factorial
{
    public static void main (String args[])
    {
        int n,x,i,j;
        long f;
        x=args.length;
        for(i=0;i<x;i++)
        {
            n=Integer.parseInt(args[i]);
            if (n<0)
                System.out.println(n + " is a Negative Number");
            else if(n==0)
                System.out.println("Factorial of\t"+ n +"\tis \t" + 1);
            else
            {
                f=1;
                for(j=1;j<=n;j++)
                    f=f*j;
                System.out.println("Factorial of\t"+ n +"\tis \t" + f);
            }
        }
    }
}
```

Critical Processing Steps

1. **Capturing Length:** The program assigns `x = args.length` to detect the total count of user inputs. This variable `x` is then used as the termination condition in the loop (`i < x`). This is a critical safety measure that ensures the program iterates exactly as many times as there are arguments, effectively preventing the dreaded `ArrayIndexOutOfBoundsException`.

2. **Conversion:** Because the command line treats all input as raw text, `args[i]` is a String. To perform mathematical operations, the program must explicitly impose a data type using `Integer.parseInt(args[i])`. This transforms the text into a numerical integer.
3. **Processing:** The outer `for` loop uses the index `i` to access each input in sequence, while the inner logic calculates and prints the factorial result.

Execution and Results

To execute this program, use the `java` command followed by the class name and the list of numbers you wish to process.

Execution Output

When running the command `java factorial 5 3 4`, the JVM processes the strings and produces the following tab-delimited output:

User Command	Generated Output
<code>java factorial 5 3 4</code>	<code>Factorial of 5 is 120</code>
	<code>Factorial of 3 is 6</code>
	<code>Factorial of 4 is 24</code>

Success in execution is only the first milestone; maintaining that success throughout your career requires writing code that adheres to professional architectural standards.

Best Practices for Readable Code

Java is fundamentally a **freeform** language. This means the compiler is indifferent to indentation or line breaks. For example, a simple print statement could technically be written as:

```
System
.
out
.
println
(
"Computer Science"
);
```

While the code above is functional, it is unreadable. Professional "Programming Style" serves as a safety rail, ensuring that your logic is clear to both your future self and your colleagues.

Readability Checklist

- **Use meaningful names** for variables, methods, and classes to convey intent.
- **Ensure a class represents** a clear, single, identifiable concept.
- **Ensure a method has** a clear, single, identifiable task (Single Responsibility Principle).
- **Use curly braces { }** consistently for all selection and looping blocks.
- **Use indentation** to visually map the structural hierarchy and nesting of your code.
- **Use comment statements** to document the "why" behind complex logic.

Finding Your Tools: PATH vs. CLASSPATH

- **Concepts:** Deconstructing the JDK folder hierarchy. Clarifying the difference between setting the **PATH** (locating binary tools like **javac** and **java** in the **/bin** directory) and **CLASSPATH** (locating compiled bytecode and libraries in the **/lib** directory) in Windows and Unix.
- **Real-World Analogy: The Tool Shed vs. The Filing Cabinet.**
 - **PATH** is the address of your tool shed. It tells your computer where the hammer (**javac**) and screwdriver (**java**) are kept so you can fetch them from any room in the house.
 - **CLASSPATH** is the drawer containing your physical design blueprints (your **.class** files and standard libraries) so the compiler knows exactly where to load your custom project sheets.

THANK YOU